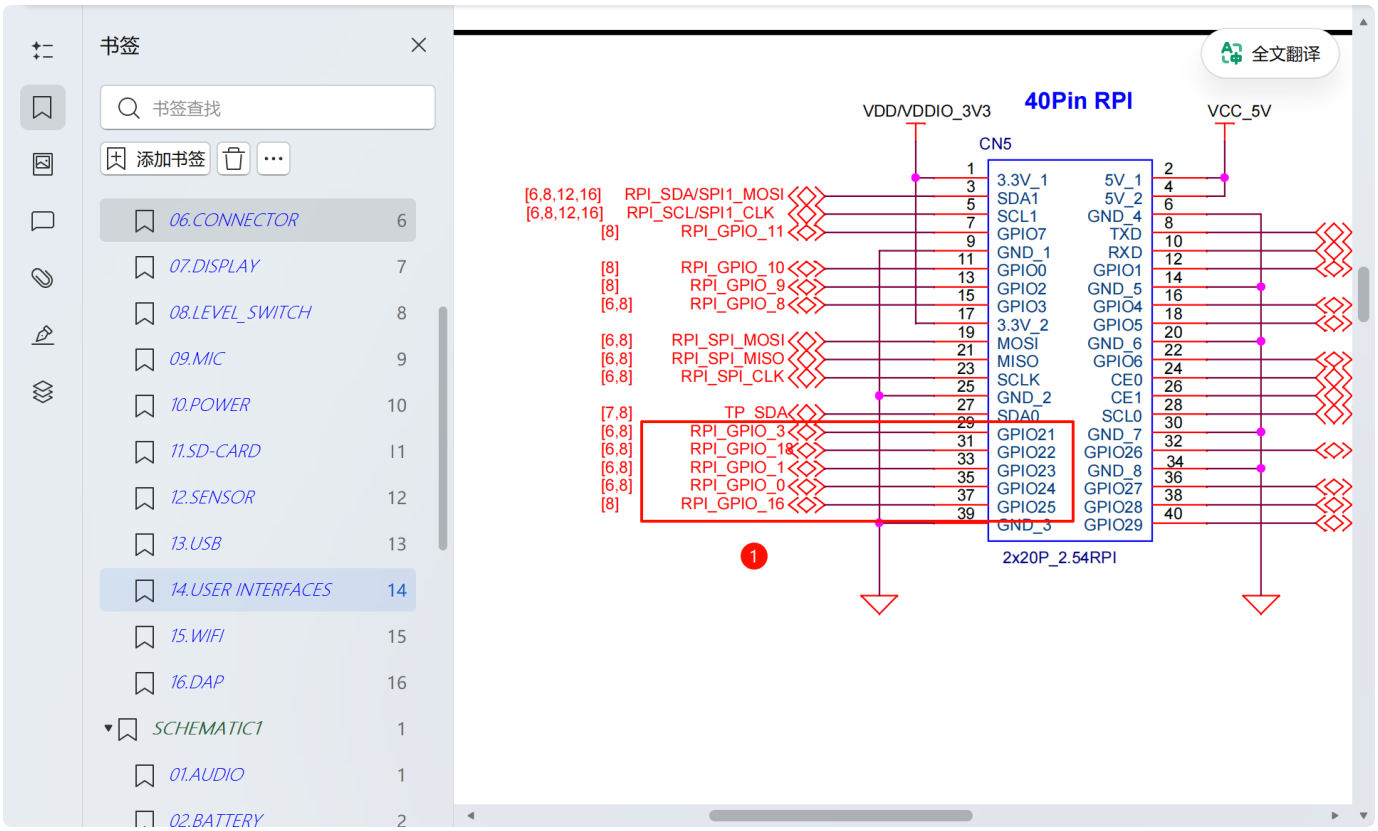
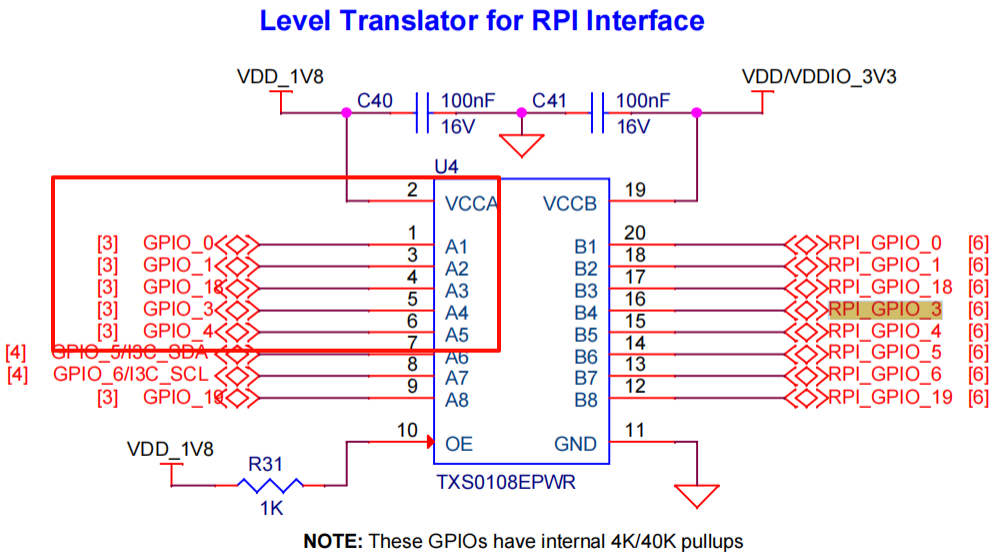
psoc-edge拓展外设配置手册

查找底板原理图，找到要配置的IO引脚

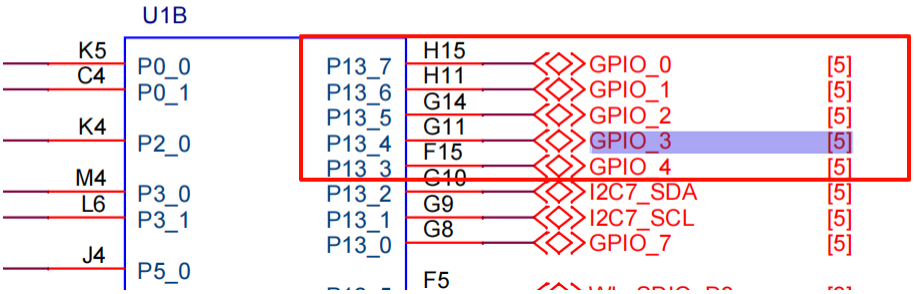


找到电源转换芯片对应的IO

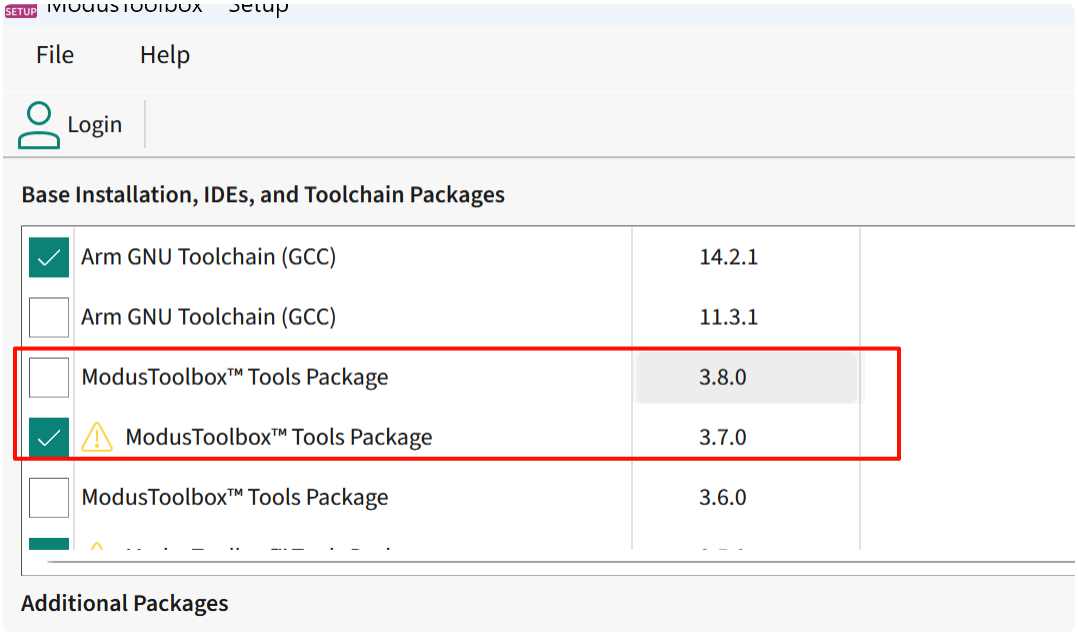
Level switch



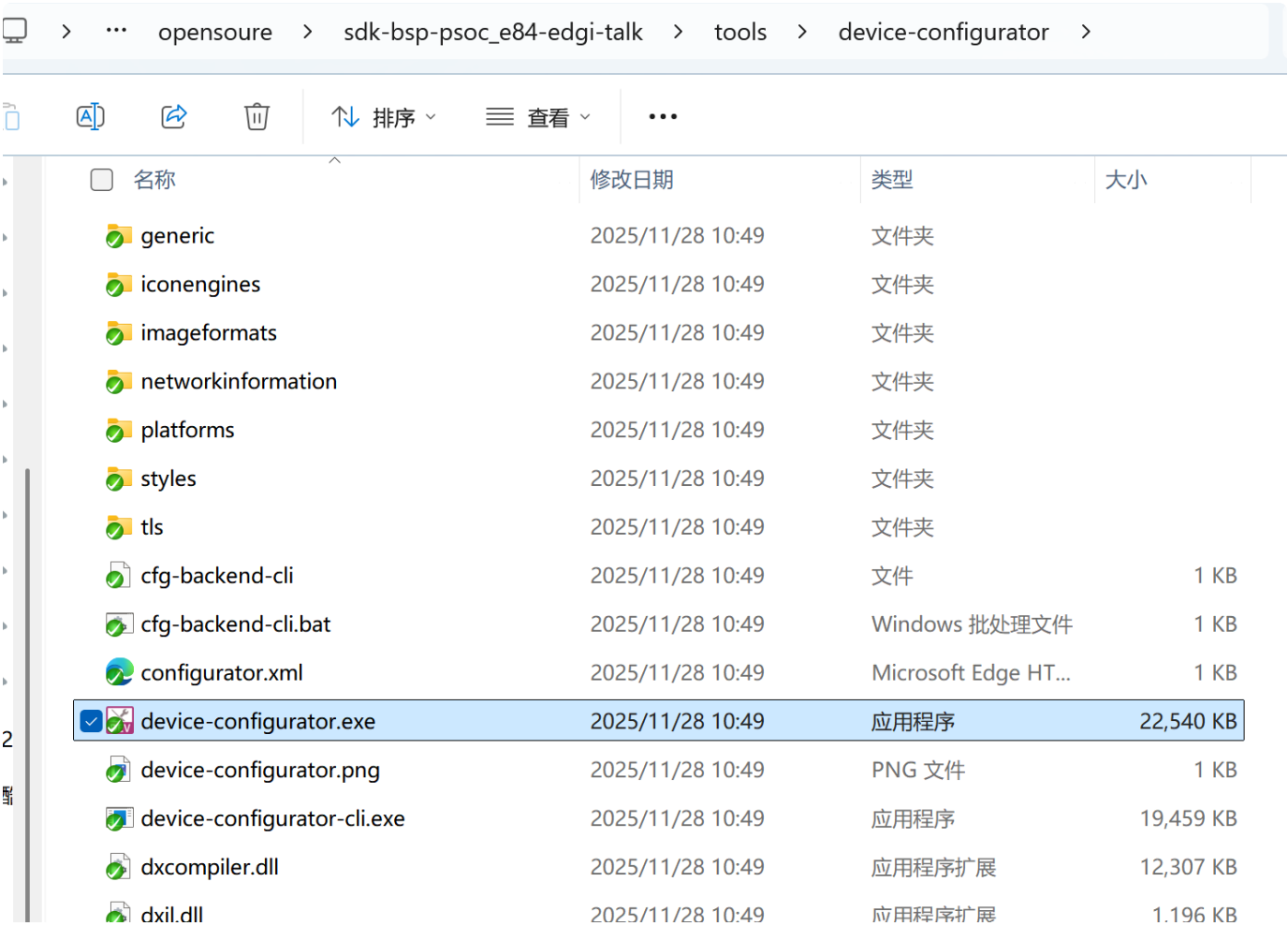
在核心板原理图找到对应的引脚定义



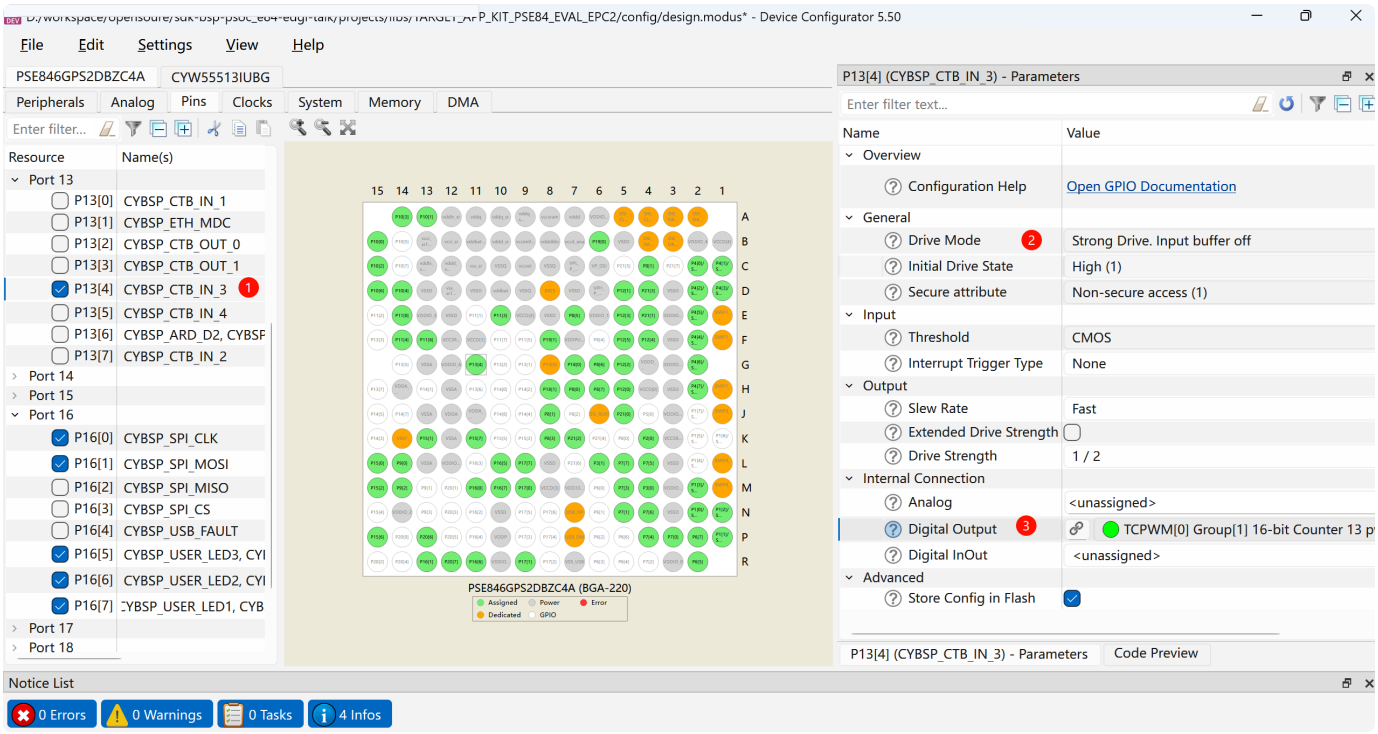
需要安装好英飞凌官方的 ModusToolbox™ Setup 软件，并安装>=3.5的Tools Package



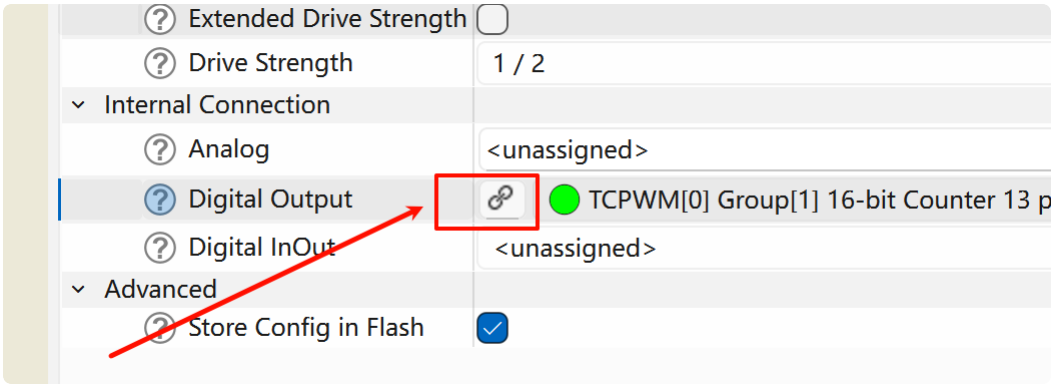
打开 device-configurator.exe 上位机，打开工程中的 `projects\libs\TARGET_APP_KIT_PSE84_EVAL_EPC2\config\design.modus` 配置文件进行配置



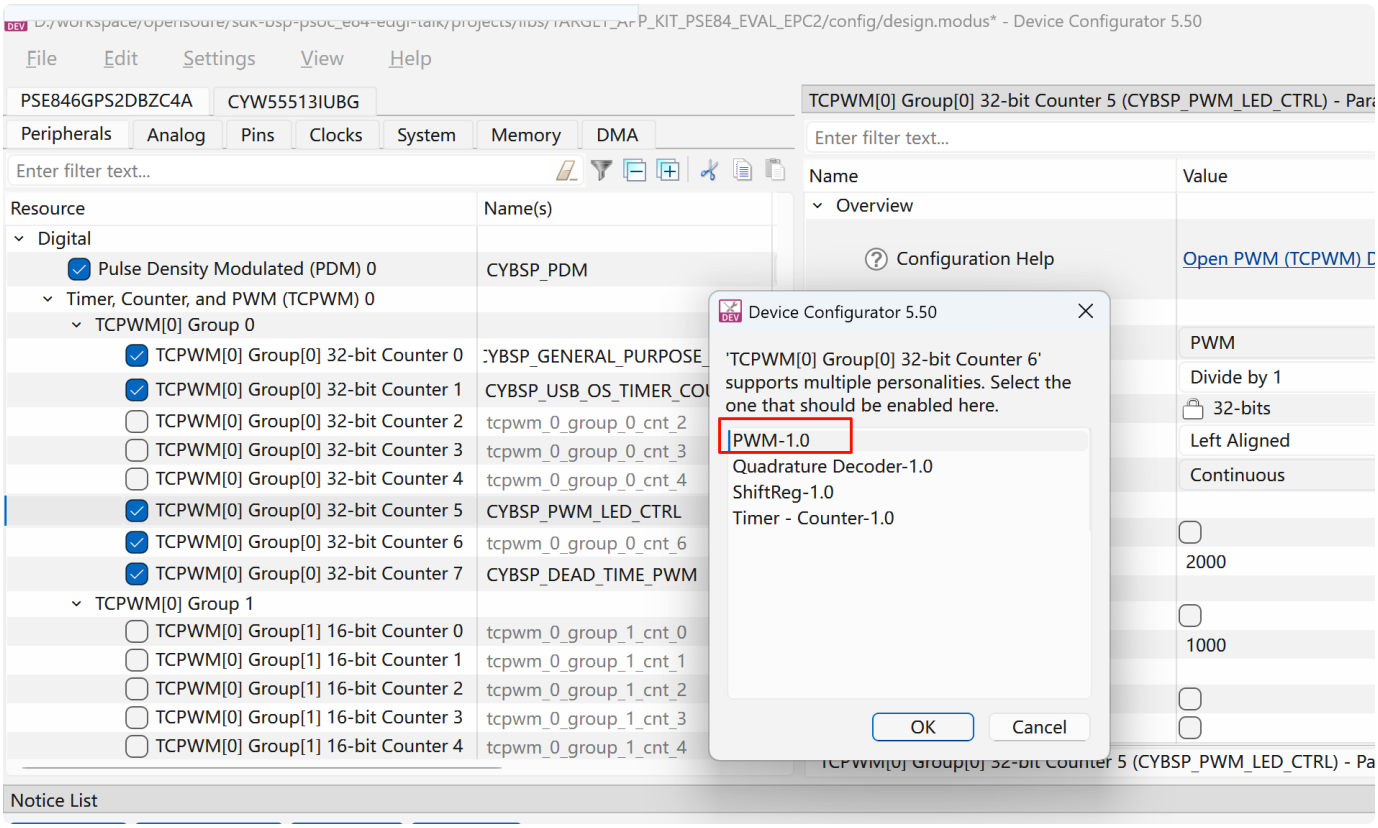
找到需要配置的IO，配置相关的工程，如下图配置 P13_4 引脚为PWM功能，需要配置IO的模式，输出的通道



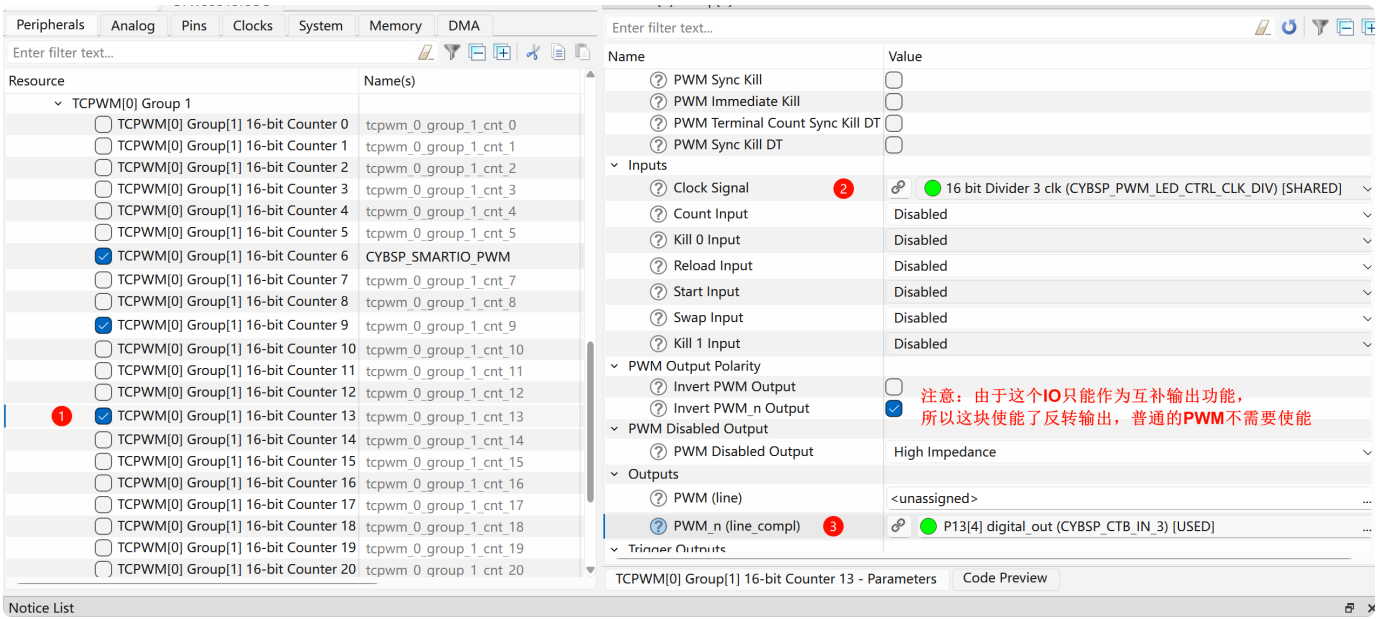
点击链接到对应的PWM配置



使能PWM通道，进行 TCPWM[0] Group[1] 的相关的配置

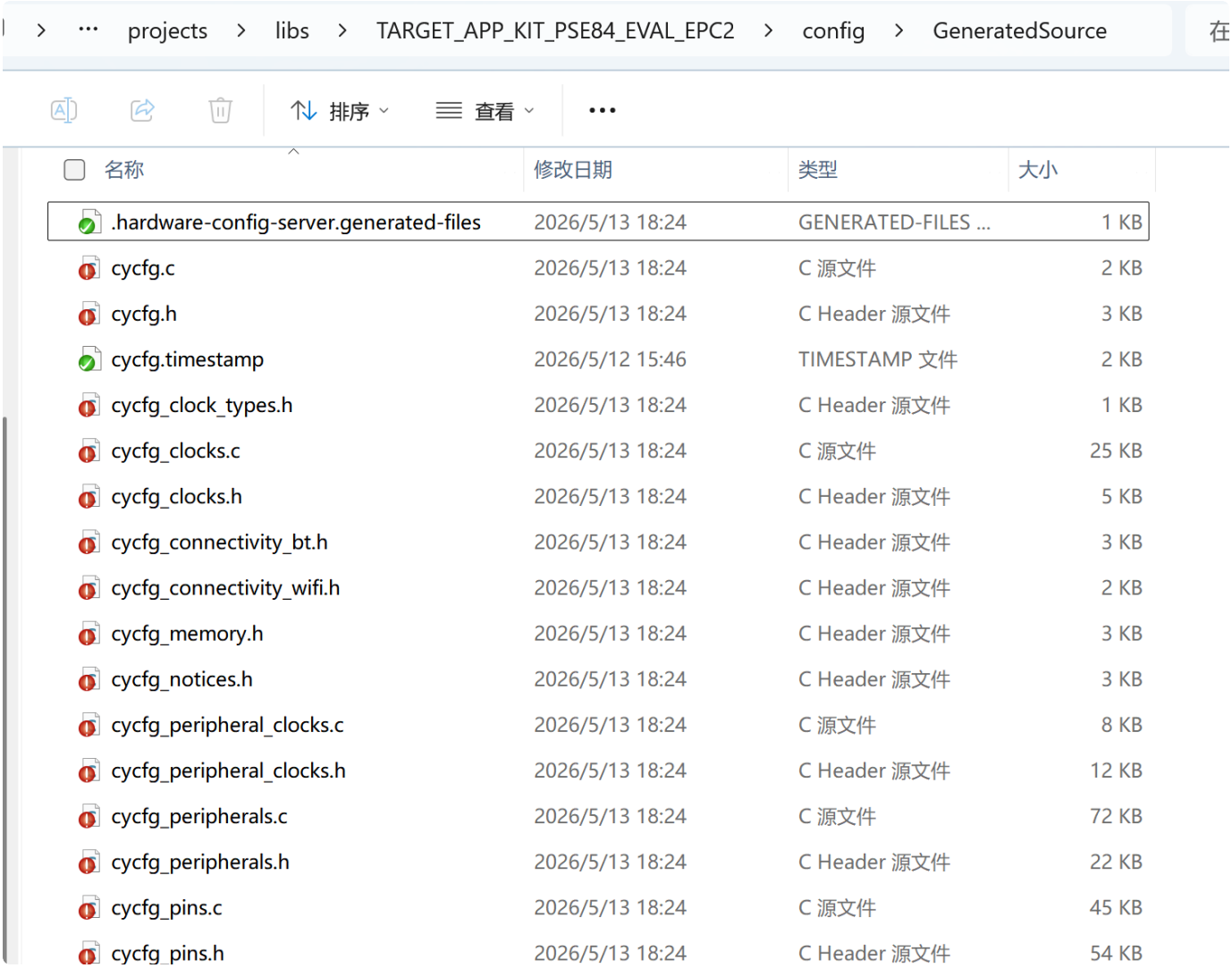


配置PWM的时钟，通道等配置

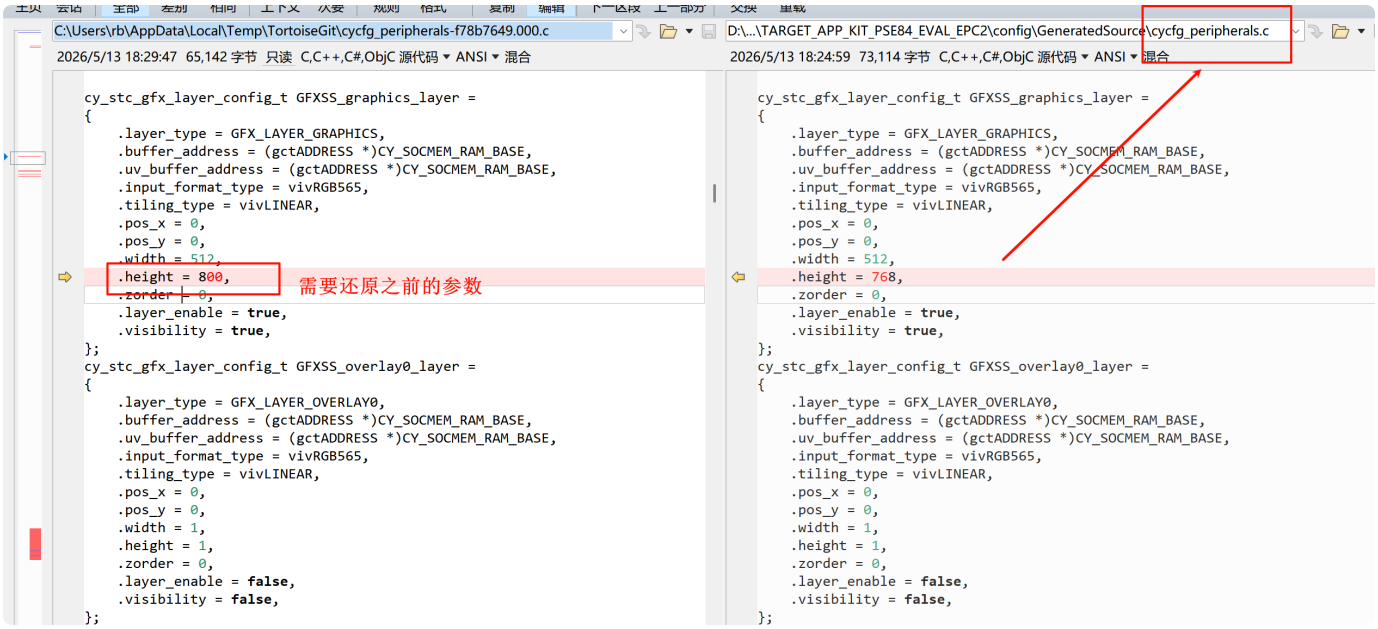


完成全部配置后，按 `Ctrl + S` 保存即可，工具会自动生成底层的代码到：

`libs\TARGET_APP_KIT_PSE84_EVAL_EPC2\config\GeneratedSource` 目录下



需要注意的是，我们还需要参考下面，还原下生成的屏幕参数配置（其余的不用管），否则后续屏幕使用将会异常：

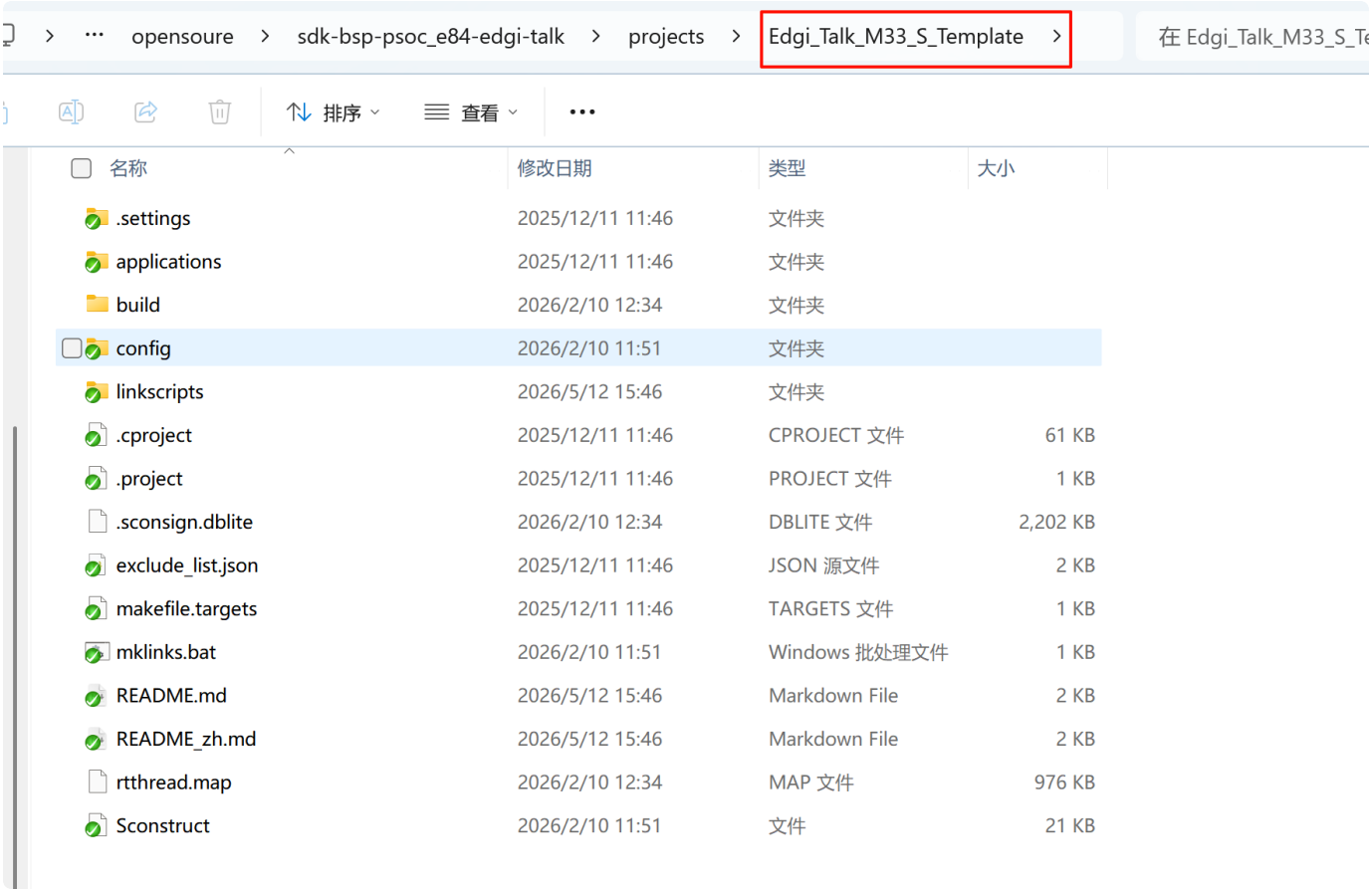


```
    .gfx_layer_config = &GFXSS_graphics_layer,
    .ovl0_layer_config = &GFXSS_overlay0_layer,
    .ovl1_layer_config = &GFXSS_overlay1_layer,
    .display_type = GFX_DISP_TYPE_DSI_DPI,
    .display_format = vivD16CFG1,
    .display_size = vivDISPLAY_CUSTOMIZED,
    .display_width = 512,
    .display_height = 800,
};
cy_stc_gfx_gpu_cfg_t GFXSS_gpu_config =
{
    .enable = true,
};
cy_stc_mipidsi_display_params_t GFXSS_mipidsi_display_params =
{
    .pixel_clock = 33448,
    .hdisplay = 512,
    .hsync_width = 2,
    .hfp = 36,
    .hbp = 40,
    .vdisplay = 800,
    .vsync_width = 2,
    .vfp = 180,
    .vbp = 10,
    .polarity_flags = 0,
};
```

```
    .gfx_layer_config = &GFXSS_graphics_layer,
    .ovl0_layer_config = &GFXSS_overlay0_layer,
    .ovl1_layer_config = &GFXSS_overlay1_layer,
    .display_type = GFX_DISP_TYPE_DSI_DPI,
    .display_format = vivD16CFG2,
    .display_size = vivDISPLAY_CUSTOMIZED,
    .display_width = 512,
    .display_height = 768,
};
cy_stc_gfx_gpu_cfg_t GFXSS_gpu_config =
{
    .enable = true,
};
cy_stc_mipidsi_display_params_t GFXSS_mipidsi_display_pa
{
    .pixel_clock = 33984,
    .hdisplay = 512,
    .hsync_width = 2,
    .hfp = 36,
    .hbp = 40,
    .vdisplay = 768,
    .vsync_width = 2,
    .vfp = 180,
    .vbp = 10,
    .polarity_flags = 0,
};
```

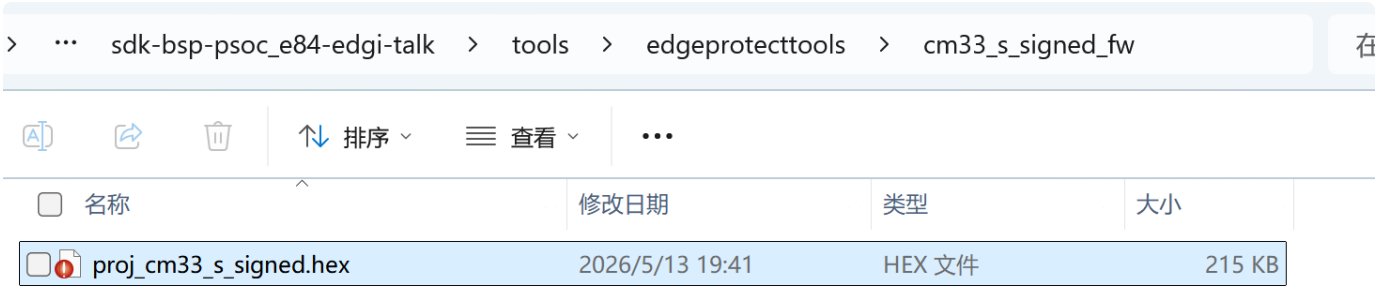
示范：如何添加额外的PWM外设

1. 重新编译 `Edgi_Talk_M33_S_Template` 工程，生成固件

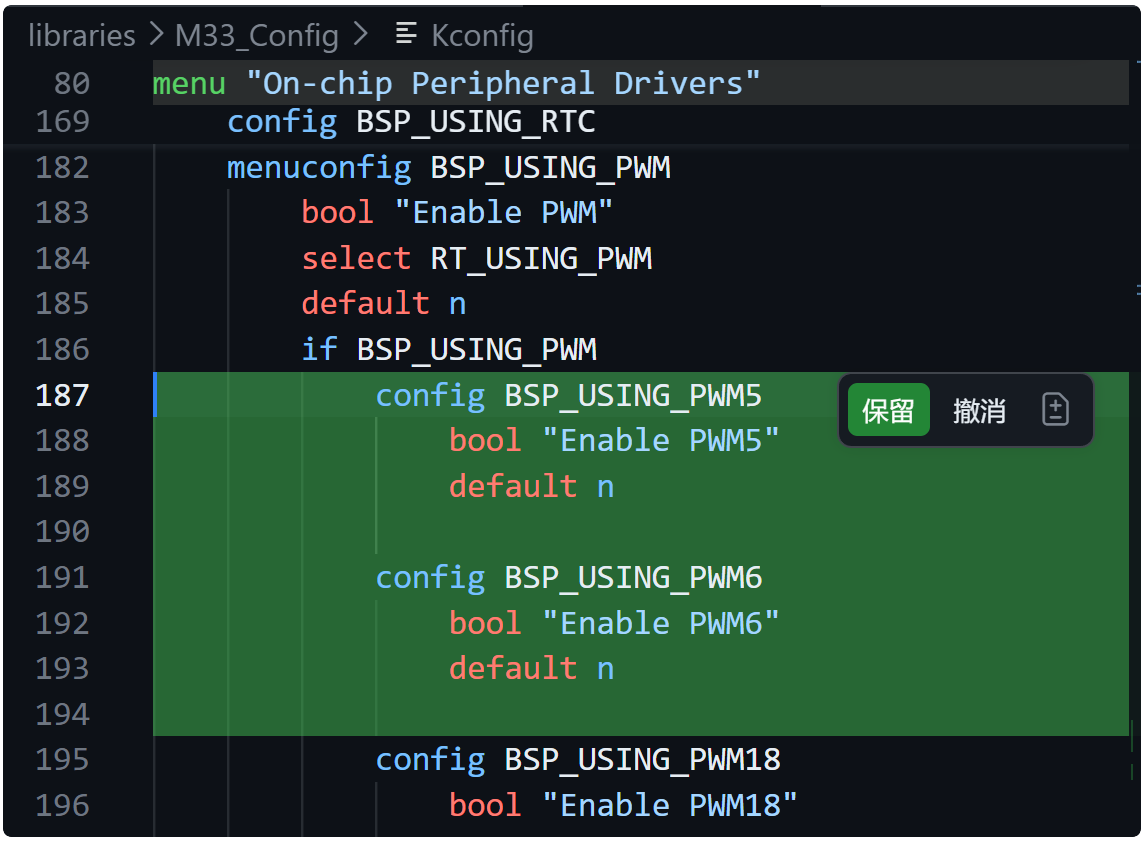


```
CC build\libs_ext\TARGET_APP_KIT_PSE84_EVAL_EPC2\cybsp.o
CC build\libs_ext\TARGET_APP_KIT_PSE84_EVAL_EPC2\system_edge.o
LINK build\rtthread.elf
D:\workspace\work\Summer\env-windows\tools\bin\..\..\tools\gnu_gcc\arm_gcc\mingw\bin\arm-none-eabi-objcopy -O ihex build\rtthread.e
 build\rtthread.hex
D:\workspace\work\Summer\env-windows\tools\bin\..\..\tools\gnu_gcc\arm_gcc\mingw\bin\arm-none-eabi-size build\rtthread.elf
D:\workspace\work\Summer\env-windows\tools\bin\..\..\tools\gnu_gcc\arm_gcc\mingw\bin\arm-none-eabi-objcopy -O binary build\rtthread
lf build\rtthread.bin
   text    data     bss     dec     hex filename
  71768    4164  131048  206980   32884 build\rtthread.elf
"D:\workspace\opensoure\sdk-bsp-psoc_e84-edgi-talk\tools\edgeprotecttools\bin\edgeprotecttools.exe" run-config -i "D:\workspace\ope
oure\sdk-bsp-psoc_e84-edgi-talk\projects\Edgi_Talk_M33_S_Template\config\boot_with_extended_boot_scons.json"
: E : INFO  : metadata_proj_cm33_s: command "sign" validation succeeded
: E : INFO  : Image saved to 'D:\workspace\opensoure\sdk-bsp-psoc_e84-edgi-talk\projects\Edgi_Talk_M33_S_Template\build\rtthread.he

: E : INFO  : metadata_proj_cm33_s: command "sign" succeeded
scons: done building targets.

(.venv) rb@RBB666 D:\workspace\opensoure\sdk-bsp-psoc_e84-edgi-talk\projects\Edgi_Talk_M33_S_Template
```

2. 拷贝 `"projects\Edgi_Talk_M33_S_Template\build\rtthread.hex"` 到 `tools\edgeprotecttools\cm33_s_signed_fw` 下，重命名为 `proj_cm33_s_signed.hex` 进行替换

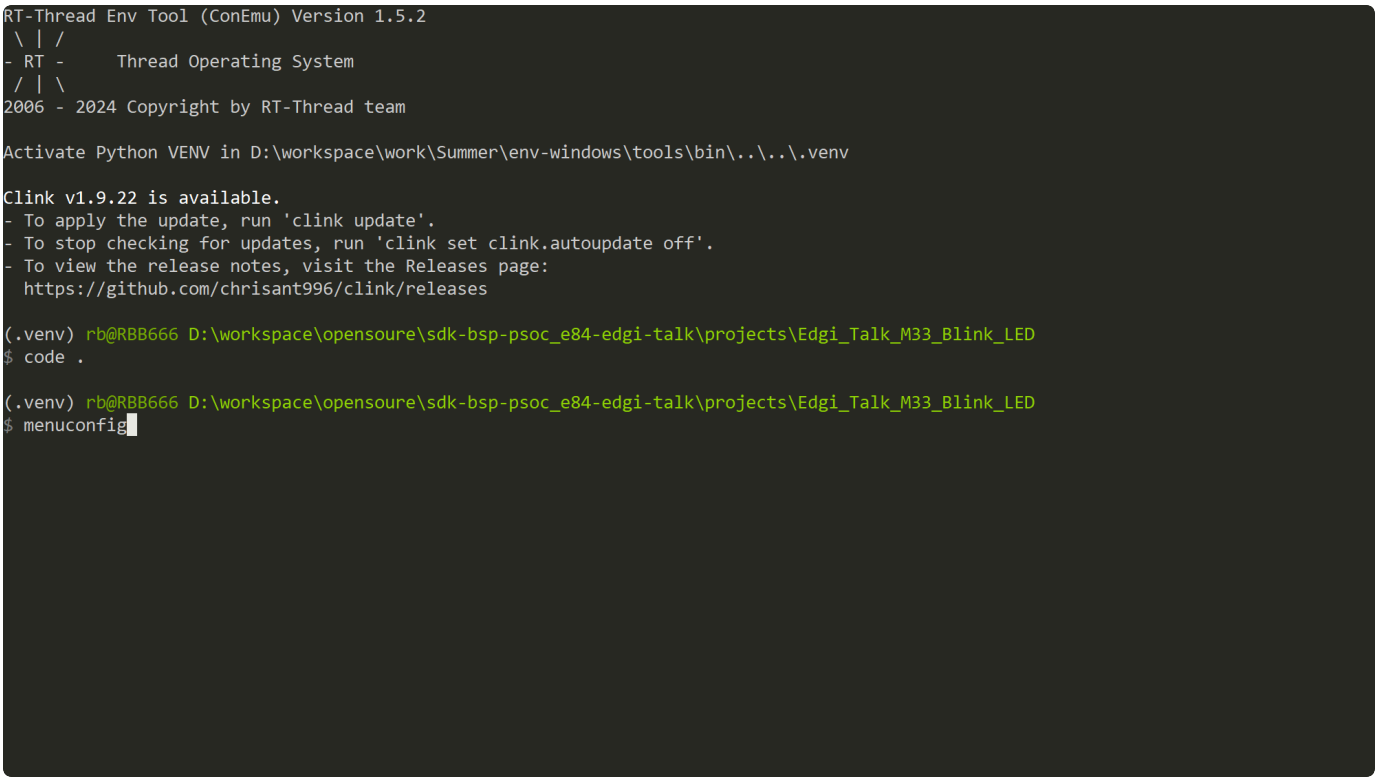


3. 打开M33对应的Kconfig文件，添加新增的PWM通道，这样在menuconfig后才能看到选项



4. 在 drv_pwm.h 中添加新的PWM配置

5. 打开env/RT-Thread Setting菜单



```
(Top) → Hardware Drivers Config→ On-chip Peripheral Drivers→ Enable PWM
                                    RT-Thread Configuration
[ ] Enable PWM5 (NEW)
[ ] Enable PWM6 (NEW)
[*] Enable PWM18 (NEW)
[ ] Enable PWM13 (NEW)

[Space/Enter] Toggle/enter  [ESC] Leave menu           [S] Save
[O] Load                    [?] Symbol info            [/] Jump to symbol
[F] Toggle show-help mode   [C] Toggle show-name mode  [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

6. 保存配置并退出

```
(Top)                          g                          s           M
                                    RT-Thread Configuration
    RT-Thread Kernel  --->
    RT-Thread Components  --->
    RT-Thread Utestcases  --->
    RT-Thread online packages  --->
    Hardware Drivers Config  --->
[ ] Using USB with CherryUSB (NEW)  ----

                                  Quit
                          Save configuration?

                          (Y)es  (N)o  (C)ancel

[Space/Enter] Toggle/enter  [ESC] Leave menu           [S] Save
[O] Load                    [?] Symbol info            [/] Jump to symbol
[F] Toggle show-help mode   [C] Toggle show-name mode  [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)
```

7. 使用scons进行编译

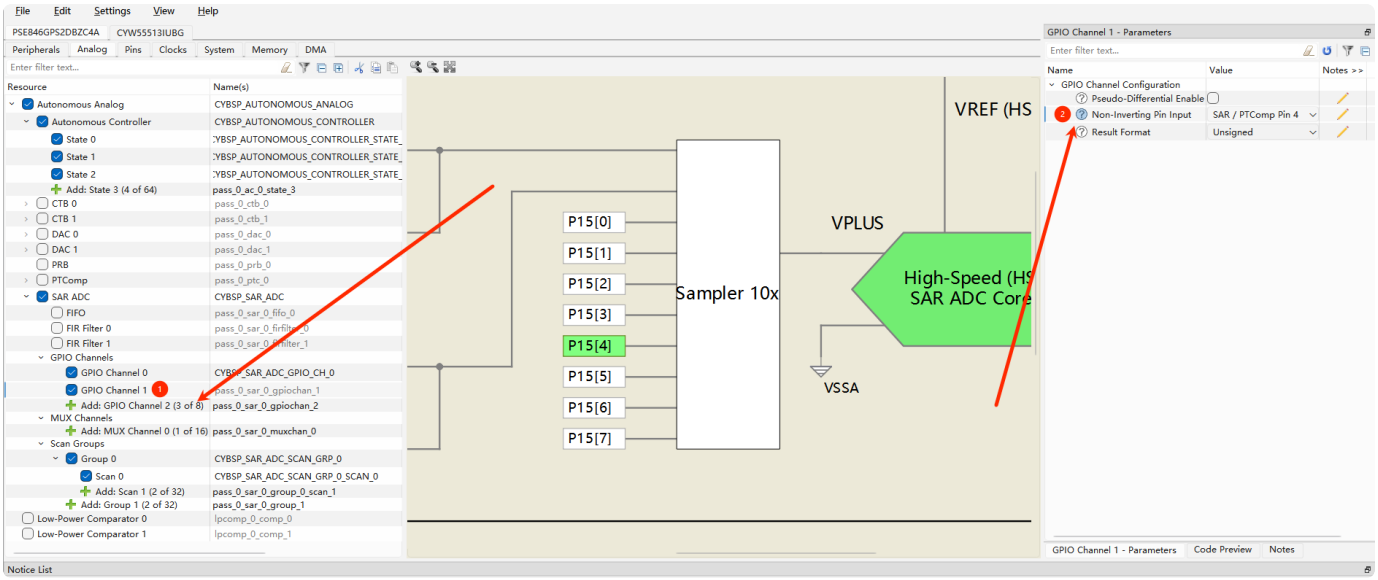
8. 使用 msh 命令进行测试：

```
msh >list device
device           type         ref count
-------- -------------------- ----------
pwm18    PWM Device           0
uart5    Character Device     0
uart2    Character Device     2
pin      Pin Device           0
msh >
msh >pwm probe pwm18  1
probe pwm18 success
msh >
msh >pwm enable pwm18  2
pwm18 channel 0 is enabled success
msh >
msh >pwm set 0 200000 150000  3
pwm info set on pwm18 at channel 0
msh >
```

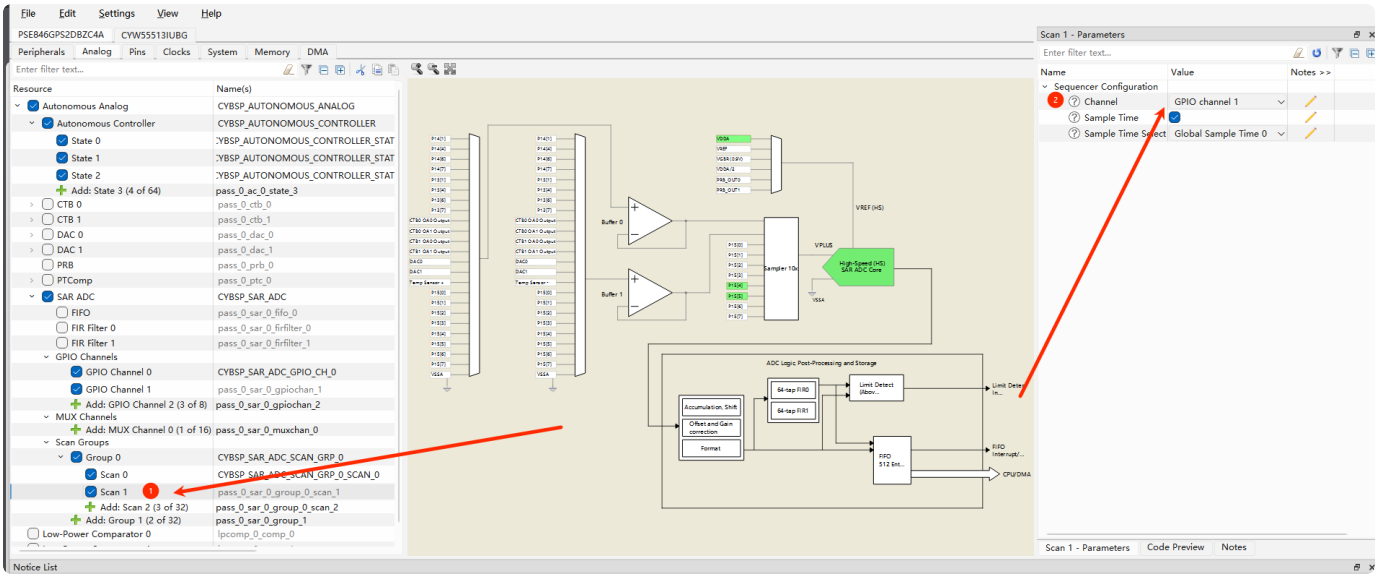
示范：如何添加额外的ADC外设

下面以配置 SPI9_SS1 为 ADC 功能为例：

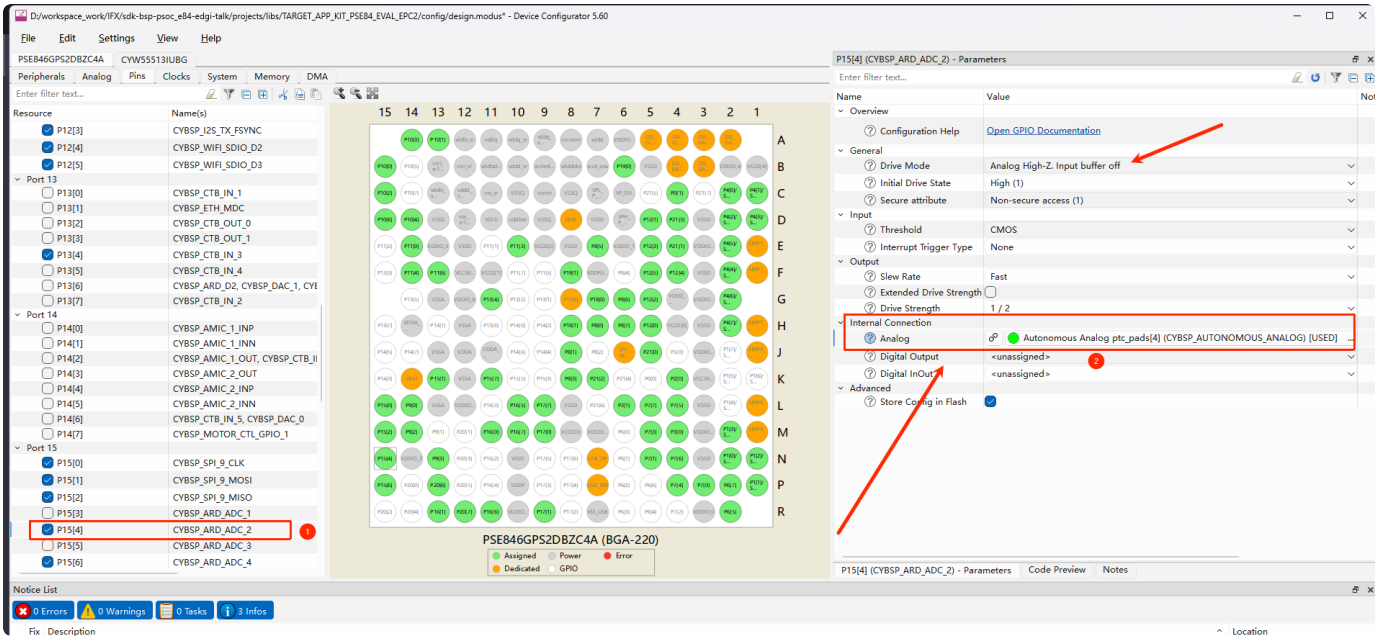
打开 device-configurator.exe 上位机，在 Analg 菜单栏中，新增一个GPIO Channel通道，然后配置IO为 Pin4，代表配置 P15[4] 引脚为ADC通道1功能。



配置 scan groups，新增一个扫描组，和上面的通道需要对应，因此这块配置为 channel 1

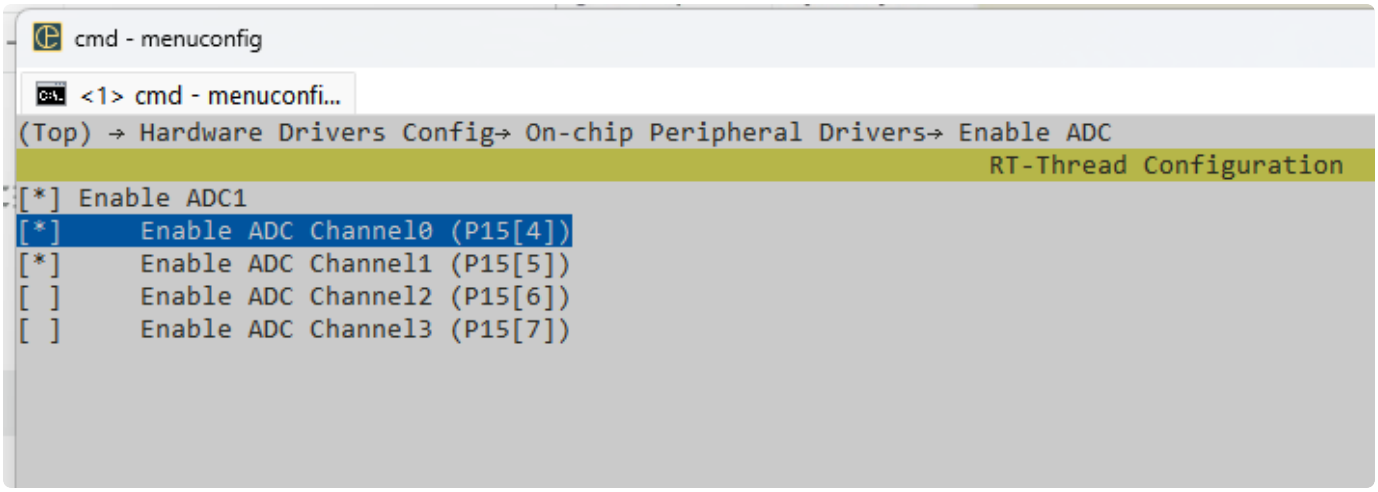


需要绑定IO引脚为具体的ADC功能



ctrl+s保存配置，生成底层代码，然后参考上面PWM的流程还原屏幕的配置代码。

在工程中使能对应的ADC通道



使用scons进行项目的构建，**注意：需要重新编译并烧录M33的工程**